TP: Estimación de π con el método de Monte Carlo

1. El problema: tirar dardos para calcular π

El ejercicio propone un experimento simulado: lanzamos N dardos al azar sobre un tablero cuadrado de lado l = 1, dentro del cual hay inscripto un círculo de radio r = 0.5. Como los dardos caen de forma uniforme (al azar) sobre el cuadrado, la proporción de dardos que caen dentro del círculo respecto del total es aproximadamente igual al cociente entre el área del círculo y el área del cuadrado.

Área del círculo = $\pi \cdot r^2 = \pi \cdot 0.25$. Área del cuadrado = $l^2 = 1$. Por lo tanto:

$$N_tablero / N \approx \pi / 4 \quad \Longrightarrow \quad \pi \approx 4 \cdot (N_tablero / N)$$

Esto significa que si contamos qué proporción de los dardos cae dentro del círculo y la multiplicamos por 4, obtenemos una estimación de π. Cuantos más dardos tiremos, mejor será la aproximación (Ley de los Grandes Números). Este tipo de técnica —usar aleatoriedad para aproximar un resultado matemático— se llama método de Monte Carlo.

Un punto (x, y) está dentro del círculo centrado en (0.5, 0.5) con radio 0.5 si cumple:

$$(x - 0.5)^2 + (y - 0.5)^2 < 0.5^2$$

El notebook resuelve el problema DOS veces: primero con NumPy puro (rápido, vectorizado) y después con Pandas (usando un DataFrame y una función fila por fila), para comparar ambos enfoques. Al final se visualiza el resultado con Seaborn.

2. Simulación con NumPy

Celda: import numpy

```
import numpy as np
```

Se importa NumPy, la librería base para trabajar con arreglos numéricos (arrays) y operaciones matemáticas vectorizadas en Python. Es la que nos va a permitir generar los números aleatorios y hacer las cuentas sin escribir bucles "for".

Celda: generación de las tiradas de dardos

```
N = 100000
coor = np.random.rand(N,2)
print(coor.shape)
```

Se define N = 100.000, la cantidad de dardos que vamos a simular. np.random.rand(N,2) genera un array de N filas y 2 columnas con números al azar extraídos de una distribución uniforme entre 0 y 1: la primera columna representa la coordenada x de cada dardo y la segunda columna la coordenada y. Como el cuadrado tiene lado 1 y sus lados van de 0 a 1, estos números ya representan directamente la posición del dardo dentro del tablero. print(coor.shape) muestra las dimensiones del array, que deberían ser (100000, 2).

Concepto clave: distribución uniforme

Una distribución uniforme es aquella donde todos los valores dentro de un rango tienen la misma probabilidad de salir sorteados. Es el modelo natural para representar "tiro dardos sin puntería": cualquier punto del tablero tiene igual chance de ser tocado.

Celda: inspección del array

```
coor
```

Simplemente muestra el contenido del array coor para ver cómo lucen los pares (x, y) generados. Es un paso exploratorio, típico al ir armando el análisis paso a paso.

Celda: ¿qué dardos caen dentro del círculo?

```
adentro = np.sqrt((coor[:,0] - 0.5)**2 + (coor[:,1] - 0.5)**2) < 0.5
```

Esta es la línea central del enfoque NumPy. coor[:,0] selecciona todos los valores de la primera columna (todas las x) y coor[:,1] todos los valores de la segunda columna (todas las y), sin necesidad de recorrer fila por fila. Se calcula la distancia de cada punto al centro del círculo (0.5, 0.5) aplicando la fórmula de distancia euclídea, y se compara con el radio 0.5. El resultado, adentro, es un array de valores booleanos (True/False): True si ese dardo cayó dentro del círculo, False si cayó afuera.

Concepto clave: vectorización

En lugar de recorrer los 100.000 puntos uno por uno con un bucle for, la operación se aplica a la vez sobre toda la columna del array. Esto se llama vectorización y es mucho más rápido que iterar en Python puro, porque NumPy ejecuta las operaciones internamente en código optimizado (C).

Celda: estimación de π

```
print(4*adentro.sum()/N)
```

adentro.sum() cuenta cuántos valores True hay en el array (es decir, cuántos dardos cayeron dentro del círculo), porque en Python/NumPy True equivale a 1 y False a 0 al sumar. Dividiendo por N se obtiene la proporción de dardos que cayeron adentro, y multiplicando por 4 se obtiene la estimación de π, siguiendo la fórmula deducida en la consigna. Con N = 100.000 el resultado debería acercarse a 3.14.

3. Pandas

El mismo experimento pero organizando los datos en un DataFrame de Pandas, para practicar las herramientas propias de esta librería: creación de columnas, funciones aplicadas fila por fila con apply, e indexado de columnas.

Celda: import pandas

```
import pandas as pd
```

Se importa Pandas, la librería para trabajar con datos tabulares (tablas tipo Excel) a través de DataFrames y Series.

Celda: creación del DataFrame

```
df = pd.DataFrame(coor, columns=['x','y'])
```

Se toma el mismo array coor generado antes con NumPy y se lo convierte en un DataFrame de Pandas, nombrando a las dos columnas 'x' e 'y'. Esto permite trabajar con los datos como una tabla, referenciando las columnas por nombre en vez de por posición.

Celda: primeras filas

```
df.head()
```

df.head() muestra las primeras 5 filas del DataFrame por defecto. Es una forma estándar de verificar rápidamente que los datos se cargaron correctamente y ver qué forma tienen las columnas x e y.

Celda: función para decidir si un punto está dentro del círculo

```
def adentro(x, y, r = 0.5):
    a = 0
    x1 = (x - 0.5)**2
    y1 = (y - 0.5)**2
    if np.sqrt(x1 + y1) < 0.5:
        a = 1
    return a
```

Se define una función propia llamada adentro que recibe una coordenada (x, y) y, opcionalmente, un radio r (con valor por defecto 0.5). Calcula la distancia al centro del círculo igual que antes, pero esta vez con lógica "fila por fila": si la distancia es menor al radio, devuelve 1 (está adentro); si no, devuelve 0 (está afuera). Es la versión "con función definida por el usuario" de la misma condición matemática usada en el enfoque NumPy, pensada para aplicarse sobre cada fila del DataFrame.

Nota: esta función reutiliza el nombre "adentro", que ya se había usado antes para el array booleano de NumPy. Al definir la función, ese nombre queda sobrescrito y ahora se refiere a la función, no al array anterior. en un notebook más prolijo convendría usar nombres distintos para evitar confusión.

Celda: prueba de la función

```
adentro(0.2, 0.05)
```

Se prueba la función con un punto puntual (0.2, 0.05) para verificar que funciona antes de aplicarla a todo el DataFrame. Ese punto está lejos del centro (0.5, 0.5), por lo que se espera que devuelva 0 (está afuera del círculo).

Celda: aplicar la función a cada fila del DataFrame

```
df['adentro'] = df.apply(lambda row: adentro(row['x'], row['y']), axis=1)
```

Esta es la línea equivalente, en el mundo Pandas, a la operación vectorizada que se hizo antes con NumPy. df.apply(..., axis=1) recorre el DataFrame fila por fila (axis=1 indica "por fila", en contraposición a axis=0 que sería "por columna") y para cada fila ejecuta la función lambda, que a su vez llama a la función adentro() con los valores de x e y de esa fila. El resultado (0 o 1) se guarda en una nueva columna llamada 'adentro'.

Concepto clave: apply vs. vectorización

df.apply con axis=1 es cómodo y expresivo, pero internamente sigue iterando fila por fila en Python, por lo que es más lento que una operación vectorizada de NumPy sobre columnas enteras (como se hizo en la sección anterior). Para datasets grandes, siempre que se pueda, conviene preferir operaciones vectorizadas antes que apply. Comparar ambos enfoques en el mismo notebook es justamente lo que permite mostrar esta diferencia.

Celda: recentrar las coordenadas

```
df['x'] = df['x'] - 0.5
df['y'] = df['y'] - 0.5
```

Se le resta 0.5 a cada columna para que el centro del círculo (y del cuadrado) pase a estar en el origen (0,0), en lugar de en (0.5, 0.5). Esto no cambia el resultado del cálculo de π (ya se usó antes para calcular 'adentro'), pero es muy útil para el paso siguiente: graficar. Centrar los datos en el origen hace que el gráfico quede simétrico y sea más fácil de interpretar visualmente.

Celda: verificar el resultado

```
df.head(5)
```

Se vuelve a mostrar las primeras filas del DataFrame, ahora ya con las columnas x e y recentradas (valores entre -0.5 y 0.5) y con la nueva columna 'adentro' agregada.

Celda: estimación de π con Pandas

```
print(4 * df.adentro.sum() / df.adentro.count())
```

Es el equivalente, con sintaxis de Pandas, al cálculo que se hizo antes con NumPy: df.adentro.sum() suma todos los 1 de la columna 'adentro' (cuántos dardos cayeron dentro del círculo) y df.adentro.count() cuenta el total de filas (equivalente a N). Multiplicando el cociente por 4 se obtiene, nuevamente, la estimación de π. El resultado debería ser muy similar al obtenido con NumPy, porque se está trabajando sobre exactamente los mismos datos.

4. Visualización del resultado con Seaborn

Celda: import seaborn

```
import seaborn as sns
```

Se importa Seaborn, una librería de visualización construida sobre Matplotlib que facilita hacer gráficos estadísticos a partir de DataFrames de Pandas, con muy poco código.

Celda: gráfico de dispersión (scatterplot)

```
sns.scatterplot(x='x', y='y', hue='adentro', data=df)
```

Se grafica cada dardo como un punto: en el eje x se usa la columna 'x' del DataFrame, en el eje y la columna 'y', y el parámetro hue='adentro' colorea los puntos según el valor de la columna 'adentro' (un

color para los que cayeron dentro del círculo y otro color para los que cayeron afuera). El resultado visual es justamente el tablero pedido en la consigna: se debería ver, a simple vista, la forma de un círculo dibujada por los puntos de un color, rodeado por los puntos del otro color que quedan en las esquinas del cuadrado. Aunque no se dibuja explícitamente el borde del círculo ni del cuadrado, sino que la nube de puntos coloreada ya deja ver esas formas.

5. Cierre

Para resumir el trabajo:

- El problema se resuelve dos veces, con dos filosofías distintas: NumPy (operaciones vectorizadas sobre arrays) y Pandas (DataFrame + función aplicada fila por fila con apply). Ambas llegan al mismo resultado.
- El núcleo matemático es siempre el mismo: generar puntos al azar, medir su distancia al centro y comparar con el radio para decidir si están dentro del círculo.
- La proporción de puntos dentro del círculo, multiplicada por 4, estima π gracias a la relación entre el área del círculo y la del cuadrado que lo contiene (método de Monte Carlo).
- Seaborn permite visualizar el resultado con una sola línea de código, coloreando los puntos según si cayeron dentro o fuera del círculo.